A Course-Based Project Report on

Complete Software Lifecycle Manager (CSLM)

Submitted to the

Department of Computer science and engineering

in partial fulfilment of the requirements for the completion of

course JAVA LAB

II YEAR BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING

Submitted by

ZAIN AHMED	25071A05N1
SHAIK AIJAAZ	25071A05K8
MANITHEJA	25071A05J5
VEDA VARDAN	25071A05E7

Under the guidance of

Dr.P.ANJUSHA

(Course Instructor)

Assistant Professor, Department of CSE, VNRVJIET

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

VALLURUPALLI NAGESWARA RAO VIGNANA JYOTHI INSTITUTE OF ENGINEERING & TECHNOLOGY

An Autonomous Institute, NAAC Accredited with 'A++' Grade, NBA

Vignana Jyothi Nagar, Pragathi Nagar, Nizampet (S.O), Hyderabad – 500 090, TS, India

2025-2026

VALLURUPALLI NAGESWARA RAO VIGNANA JYOTHI INSTITUTE OF ENGINEERING AND TECHNOLOGY

An Autonomous Institute, NAAC Accredited with 'A++' Grade, NBA Accredited for CE, EEE, ME, ECE, CSE, EIE, IT B. Tech Courses, Approved by AICTE, New Delhi, Affiliated to JNTUH, Recognized as "College with Potential for Excellence" by UGC, ISO 9001:2015 Certified, QS I GUAGE Diamond Rated

Vignana Jyothi Nagar, Pragathi Nagar, Nizampet(SO), Hyderabad-500090, TS, India

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to certify that the project report entitled **"Complete Software Lifecycle Manager (CSLM)"** is a bonafide work done under our supervision and is being submitted by **ZAIN AHMED (25071A05N1), SHAIK AIJAAZ (25071A05K8), MANITHEJA (25071A05J5), VEDA VARDAN (25071A05E7)** in partial fulfilment for the award of the degree of **II Year Bachelor of Technology** in **Computer Science and Engineering** , of the VNRVJIET, Hyderabad during the academic year 2025-2026.

Dr.P.Anjusha

Assistant Professor

Dr.V.Baby

Associate Professor &

HOD Department of CSE

Department of CSE

Course based Projects Reviewer

VALLURUPALLI NAGESWARA RAO VIGNANA JYOTHI INSTITUTE OF ENGINEERING AND TECHNOLOGY

An Autonomous Institute, NAAC Accredited with 'A++' Grade,

Vignana Jyothi Nagar, Pragathi Nagar, Nizampet(SO), Hyderabad-500090, TS, India

DEPARTMENT OF COMPUTER SCIENCE AND TECHNOLOGY

DECLARATION

We declare that the course based project work entitled **"Complete Software Lifecycle Manager (CSLM)"** submitted in the Department of Computer Science and Engineering, Vallurupalli Nageswara Rao Vignana Jyothi Institute of Engineering and Technology, Hyderabad, in partial fulfilment of the requirement for the award of the Degree of **II Year Bachelor of Technology in COMPUTER SCIENCE AND ENGINEERING** is a bonafide record of our own work carried out under the supervision of **Dr.P.Anjusha , department of CSE, VNRVJIET**. Also, we declare that the matter embodied in this thesis has not been submitted by us in full or in any part thereof for the award of any degree/diploma of any other institution or university previously.

Place: Hyderabad.

ZAIN AHMED	SHAIK AIJAAZ	MANITHEJA	VEDA VARDAN
(25071A05N1)	(25071A05K8)	(25071A05J5)	(25071A05E7)

ACKNOWLEDGEMENT

We express our deep sense of gratitude to our beloved President, Sri. D. Suresh Babu, VNR Vignana Jyothi Institute of Engineering & Technology for the valuable guidance and for permitting us to carry out this project.

With immense pleasure, we record our deep sense of gratitude to our beloved Principal, Dr. C.D Naidu, for permitting us to carry out this project.

We express our deep sense of gratitude to our beloved Professor **Dr.V.Baby, Associate Professor and Head, Department of CSE & CSBS**, VNR Vignana Jyothi Institute of Engineering & Technology, Hyderabad-500090 for the valuable guidance and suggestions, keen interest and through encouragement extended throughout the period of project work.

We take immense pleasure to express our deep sense of gratitude to our beloved Guide, **Dr.P.Anjusha, Assistant Professor in Computer Science and Engineering**, VNR Vignana Jyothi Institute of Engineering & Technology, Hyderabad, for his/her valuable suggestions and rare insights, for constant source of encouragement and inspiration throughout our project work.

We express our thanks to all those who contributed for the successful completion of our project work.

1.ZAIN AHMED (25071A05N1)
2.SHAIK AIJAAZ (25071A05K8)
3.MANITHEJA (25071A05J5)
4.VEDA VARDAN (25071A05E7)

TABLE OF CONTENTS

S.NO.	CONTENTS	PAGE NO.
1.	ABSTRACT	1
2.	INTRODUCTION	2
3.	LITERATURE REVIEW	3
4.	RESEARCH GAP	4
5.	EXISTING SYSTEM	5
6.	PROPOSED SYSTEM	6
7.	METHODOLOGY	7-8
8.	ARCHITECTURE AND IMPLEMENTATION	9-20
9.	CONCLUSIONS	21
10.	FUTURE SCOPE	22
11.	REFERRENCES	23

ABSTRACT

This project focuses on designing and implementing the Complete Software Lifecycle Manager (CSLM), an integrated, multi-tenant software engineering and DevSecOps platform that unifies project planning, Agile sprint execution, Kanban workflow orchestration, defect triage, in-browser code editing, version control simulation, collaborative whiteboarding, real-time communication, and technical recruitment inside a single cohesive application.

Conceptually structured as a five-layer software engineering architecture, CSLM processes engineering workflows through deterministic state transitions: authentication and role verification, multi-tenant project hydration, task and defect lifecycle management, snapshot-guarded AI code refactoring, and dual-mode persistence across Supabase PostgreSQL and browser-native DynamicStorage.

The implementation integrates React 19.2.8, Vite 8.2.0, Tailwind CSS 4.3.3, Monaco Editor (@monaco-editor/react), Excalidraw (@excalidraw/excalidraw), WebRTC/LiveKit media rooms, Socket.IO real-time messaging, Electron 35.7.5, and a 21-table PostgreSQL schema fortified with Row-Level Security (RLS). Domain engines—including the Kanban AI Workflow evaluator (kanbanAiWorkflow.js) and the OpenCode Developer Agent (openCodeAgent.js)—ensure that automated task progression and AI-assisted code edits remain auditable, syntax-validated, and subject to human review gates.

A responsive RotationalWorkspace interface serves as the unified frontend shell, enabling Administrators, Managers, Developers, and Testers to plan sprints, write and diff source code, track real defect Mean Time To Resolution (MTTR), collaborate on architecture diagrams, and manage private developer workspaces with zero cross-tenant or cross-user data leakage.

INTRODUCTION

In modern software engineering organizations and academic laboratories, development teams rely on a fragmented collection of disconnected tools—issue trackers such as Jira for Kanban boards, VS Code for source code editing, GitHub for pull requests and CI/CD pipelines, Slack for team chat, Zoom for video standups, Excalidraw for architecture diagrams, and separate portals for recruitment. Switching constantly between these isolated tools causes severe context loss, stale task states, and security blind spots.

In Complete Software Lifecycle Manager (CSLM), every stage of the software lifecycle is unified within a single multi-tenant workspace governed by a centralized state facade (AppContext.jsx) and a deterministic 6-state boot machine (BOOTING, AUTHENTICATING, AUTHENTICATED, PROJECT_LOADING, READY, and AUTH_ERROR). When a developer creates or moves a Kanban task, CSLM automatically synchronizes structured .tasks/<id>.json and .tasks/<id>.html artifacts directly into the project code tree. Similarly, merging a Pull Request or passing a CI/CD pipeline automatically advances linked tasks across lifecycle stages.

The primary objective of CSLM is to provide a production-grade, RBAC-enforced engineering platform where every module—from PostgreSQL Row-Level Security (RLS) and real Mean Time To Resolution (MTTR) bug analytics to the 8-step OpenCode AI code refactoring agent—operates transparently, securely, and reliably both in cloud-connected Supabase environments and in zero-configuration offline laboratory sandboxes.

LITERATURE REVIEW

A rigorous examination of existing software lifecycle management platforms, database security models, and AI-assisted programming environments establishes the theoretical and architectural foundation for the Complete Software Lifecycle Manager (CSLM). The review traces the evolution from monolithic Application Lifecycle Management (ALM) suites to modern local-first, AI-augmented DevSecOps workspaces:

- **Agile Lifecycle & Kanban Foundations:** Early ALM platforms such as IBM Rational and HP Quality Center enforced rigid waterfall traceability but suffered from high configuration overhead. Modern Agile frameworks—formalized in the Scrum Guide and Anderson's Kanban methodology—introduced iterative sprints and visual work-in-progress columns (To Do, In Progress, Review, Done), yet commercial trackers rarely synchronize board state directly with source code artifacts.
- **Multi-Tenant Database Security & Row-Level Security (RLS):** Cloud-native SaaS platforms require strict tenant isolation so members of one engineering squad cannot view or mutate another team's projects or private notes. PostgreSQL Row-Level Security (RLS) combined with SECURITY DEFINER helper functions (is_admin() and current_user_team_id()) provides kernel-level access enforcement directly inside the database engine.
- **Browser-Based IDE & Safe AI Code Assistants:** The Microsoft Monaco Editor engine enabled full-featured syntax highlighting and code intelligence inside web browsers. While recent Large Language Model (LLM) coding assistants accelerate refactoring, direct unguarded buffer replacement often introduces syntax errors or unbalanced delimiters, motivating CSLM's 8-step OpenCode snapshot, working-copy validation, and diff-review pipeline.
- **Real-Time Collaboration (WebRTC, Socket.IO & Vector Whiteboarding):** Distributed software teams require synchronous communication channels alongside code and task views. WebRTC MediaDevices (getUserMedia and getDisplayMedia), Socket.IO event rooms, and @excalidraw/excalidraw vector canvases allow real-time video standups, typing indicators, and debounced architectural diagramming within the browser.
- **Local-First Dual-Persistence Architectures:** As established by Kleppmann et al. in local-first software research, resilient applications should function seamlessly offline while synchronizing with cloud databases when available. CSLM realizes this through its Repository adapter (src/lib/repository.js) and DynamicStorage engine (src/lib/supabase.js), bridging snake_case PostgreSQL tables with camelCase browser state.

RESEARCH GAP

The current landscape of academic and commercial software project management systems exhibits several critical architectural, governance, and usability gaps that CSLM directly addresses:

- **Fragmentation Across Disconnected Engineering Toolchains:** A critical deficit exists in conventional workflows where Kanban boards, code editors, bug trackers, whiteboards, chat channels, and recruitment pipelines exist as separate applications. Tasks remain passive database rows rather than automatically generating structured .tasks/<id>.json and .tasks/<id>.html engineering artifacts inside the repository.
- **Lack of Governance and Safety in AI Code & Task Automation:** Most prototype AI integrations overwrite editor buffers directly or move Kanban cards without inspecting verifiable engineering evidence. There is a clear need for deterministic rule engines (evaluateTaskProgression) that verify commits, subtasks, PR status, and blocking bugs while enforcing human review gates before marking tasks Done.
- **Unenforced Defect Resolution & Missing Real MTTR Telemetry:** In many issue trackers, any team member can prematurely mark a high-severity defect as 'Fixed', and analytics dashboards frequently display hardcoded placeholder numbers instead of computing true Mean Time To Resolution (MTTR) from bug creation (createdAt) to verified resolution (resolvedAt).
- **Multi-Tenant Cloud Security vs. Offline Laboratory Portability:** Applications built exclusively on cloud APIs fail to boot in offline academic labs when credentials are absent, whereas localStorage-only prototypes lack formal SQL schemas, foreign key cascades, and Row-Level Security (RLS). CSLM bridges this gap with a 21-table RLS-hardened Supabase schema paired with an automatic DynamicStorage fallback.

EXISTING SYSTEM

Conventional software engineering workflows rely on disparate third-party tools that operate in isolation, introducing synchronization delays, security inconsistencies, and fragile external dependencies. These shortcomings, detailed below, highlight the necessity for CSLM's unified lifecycle architecture:

- **Monolithic or Disconnected Tool Silos:** Teams must manually copy task IDs, branch names, and defect statuses across separate issue trackers, Git portals, code editors, and chat apps. Updating a pull request or pipeline does not automatically transition the corresponding Kanban task or update in-repo task documentation.
- **Permissive Access Control and Cross-Team Leakage:** Without strict multi-tenant scoping (projectEligibleUsers) and database-level Row-Level Security, users from one team can accidentally be assigned tasks in another team's project, or unauthorized users can mark critical defects as 'Fixed' without being the assigned engineer or an Administrator.
- **Unsafe In-Place AI Code Modifications:** Standard AI chat plugins require manual copy-pasting or overwrite active files without capturing recovery snapshots, checking for balanced braces/brackets/parentheses, validating JSON syntax, or detecting unresolved Git merge conflict markers.
- **Fragile External Service Dependencies:** When an external mail server, WebSocket chat relay, or cloud LLM gateway is unreachable, traditional web clients crash or hang indefinitely rather than probing health with short timeouts and falling back gracefully to isolated local development storage.
- **Absence of Private Developer Workspace Isolation:** Project management suites focus solely on shared team boards and provide no strictly isolated personal command center for individual developers to maintain private notes, scratchpad code files, focus timers, and personal AI sessions on shared workstations.

PROPOSED SYSTEM

The proposed system—Complete Software Lifecycle Manager (CSLM)—introduces a five-layer, multi-tenant architecture built with React 19, Vite 8, Supabase PostgreSQL, and Electron 35. It rigorously divides software lifecycle operations into modular, verifiable layers:

- **Presentation & RotationalWorkspace Shell Layer (src/App.jsx, src/components/*):** Governed by a 6-state boot machine (BOOTING -> AUTHENTICATING -> AUTHENTICATED -> PROJECT_LOADING -> READY / AUTH_ERROR), this layer provides lazy-loaded views (React.lazy), ModuleErrorBoundary protection, a Cmd/Ctrl+K CommandPalette, global keyboard shortcuts, and 3000ms auto-dismissing notifications.
- **Application Orchestration Facade Layer (src/context/AppContext.jsx):** Exposes the unified useApp() context hook across all 16 views. It enforces team and project scoping, filters assignee dropdowns via projectEligibleUsers, executes canonical moveTask() transitions with actor attribution, and auto-generates .tasks/<id>.json and .tasks/<id>.html files inside codeFiles.
- **Domain Services & Safe AI Engines Layer (src/domain/*):** Encapsulates pure business rules: authService.js (Supabase JWT Auth + 6 RBAC demo personas + sanitizeUser()), kanbanAiWorkflow.js (evidence-based task progression with mandatory human review before Done), openCodeAgent.js (8-step snapshot, working copy, syntax validation, and diff review), moxMailAdapter.js, and aiProvider.js.
- **Dual-Persistence Repository & RLS Database Layer (src/lib/*, supabase/schema.sql):** Repository (repository.js) translates camelCase JavaScript objects to snake_case SQL columns while preserving JSONB fields. It persists to a 21-table Supabase PostgreSQL database protected by Row-Level Security (RLS) and transparently falls back to DynamicStorage (CSLM_DB_* and CSLM_PERSONAL_<userId>_*) when offline.

Advantages of the Proposed System:

- Deterministic 6-state startup lifecycle eliminating authentication and project hydration race conditions.
- Database-enforced RBAC and RLS across 21 tables, including a WITH CHECK constraint ensuring only an Admin or the assigned developer can mark a bug as 'Fixed'.
- Snapshot-guarded OpenCode AI refactoring in Monaco Editor with automatic delimiter/JSON syntax validation and 1-click rollback.
- Integrated Excalidraw whiteboarding (400ms debounced save + PNG export), WebRTC/LiveKit video rooms, Socket.IO real-time chat, and health-probed Mox developer mail.
- Authentic engineering analytics computing real Mean Time To Resolution (MTTR) from defect timestamps and weighted 60/20/20 candidate skill matching in the Talent module.

METHODOLOGY

The project methodology is structured as a seven-step software engineering pipeline, rigorously governing authentication, multi-tenant data hydration, Agile task progression, and AI-assisted code safety:

1. **Session Bootstrapping and Multi-Tenant RBAC Authentication:** Upon launch, App.jsx transitions through the 6-state machine and invokes AuthService.restoreSession(). Users authenticate via Supabase Auth (JWT) or select one of 6 role-based sandbox personas (Admin, Manager, Developer, Tester across Team Alpha t1 and Team Beta t2). sanitizeUser() strips sensitive credential fields before storing session state.
2. **Project Hydration and Bidirectional Repository Synchronization:** AppContext fetches project-scoped entities through Repository.getAll(tableName, fallback, filters). The adapter applies server-side .eq() and .in() filters, converts PostgreSQL snake_case columns to JavaScript camelCase (preserving JSONB columns such as history, subtasks, elements, permissions, and metadata), and caches state in DynamicStorage.
3. **Sprint Planning, Task Assignment, and Automatic .tasks/ Artifact Generation:** Managers create sprints and assign tasks/bugs exclusively to team members returned by projectEligibleUsers. Moving or creating a task via moveTask() records actor attribution (lastMovedBy/From/To/At) and automatically writes .tasks/<taskId>.json and a standalone .tasks/<taskId>.html viewer into the project's code_files tree.
4. **Evidence-Based Kanban AI Task Progression (kanbanAiWorkflow.js):** When evaluating a task, evaluateTaskProgression() checks linked Git commits, subtask completion ratios, Pull Request merge state, CI/CD pipeline runs, and open High/Critical bug blockers on the task's branch. Advancing from Review to Done enforces requireHumanReviewForDone: true, issuing REQUEST_REVIEW until approved by a human.
5. **Monaco Code Authoring and OpenCode 8-Step Safe AI Refactoring:** Inside CodeEditorView.jsx, invoking the OpenCode Developer Agent triggers an 8-step safety pipeline: (1) createEditorRecoverySnapshot(), (2) createWorkingCopy(), (3) buildProjectAIContext(), (4) executeOpenCodeTask() via gateway or local rule engine, (5) validateWorkingCopy() checking JSON and bracket/brace/paren balance, (6) computeFileDiff(), (7) user diff review & approval, and (8) optional 1-click snapshot rollback.
6. **Collaborative Whiteboarding, Real-Time Chat, Video Rooms, and Mox Mail:** Whiteboard.jsx embeds @excalidraw/excalidraw inside WhiteboardErrorBoundary with 400ms debounced persistence and PNG export. ChatView connects to scripts/chat-server.mjs (Socket.IO :3001) with Local Mode fallback. VideoCallView manages WebRTC camera/screen streams and room slugs, while moxMailAdapter probes VITE_MOX_SERVER_URL/api/health (2s timeout) before using isolated per-user local mailboxes.

7. **Evaluation and Automated Verification:** System correctness and security are quantitatively verified using four automated Node.js test suites (verify_repairs.mjs: 15/15 passed; verify_second_master_repair.mjs: 9/9 passed; verify_master_spec.mjs: 10/10 passed; verify_personal_and_ai.mjs: 8/8 passed) alongside static AST linting via Oxlint (0 errors across 74 source files).

8. Technologies used:

Category	**Tools / Technologies**
Frontend Framework & Bundler	React 19.2.8, React DOM 19.2.8, Vite 8.2.0 (@vitejs/plugin-react 6.0.4)
UI Styling & Motion	Tailwind CSS 4.3.3, Framer Motion 13.0.0, Lucide & Carbon Icons, Three.js 0.185.1
Database & Cloud Backend	Supabase PostgreSQL (@supabase/supabase-js 2.112.4, 21 RLS-Protected Tables)
Offline Persistence Adapter	DynamicStorage (Browser localStorage with Tenant & Per-User Key Isolation)
In-Browser Code IDE & AI	Monaco Editor (@monaco-editor/react 4.7.0), OpenCode 8-Step Agent, Gemini / FreeLLMAPI
Collaboration & Real-Time	Excalidraw 0.18.1, Socket.IO 4.8.4, WebRTC MediaDevices & LiveKit 2.22.3, Mox Mail
Desktop Shell & Static Verification	Electron 35.7.5 (electron/main.cjs), Oxlint 1.75.0, Node.js ES Module Test Suites (.mjs)

ARCHITECTURE AND IMPLEMENTATION OF THE PROPOSED SYSTEM

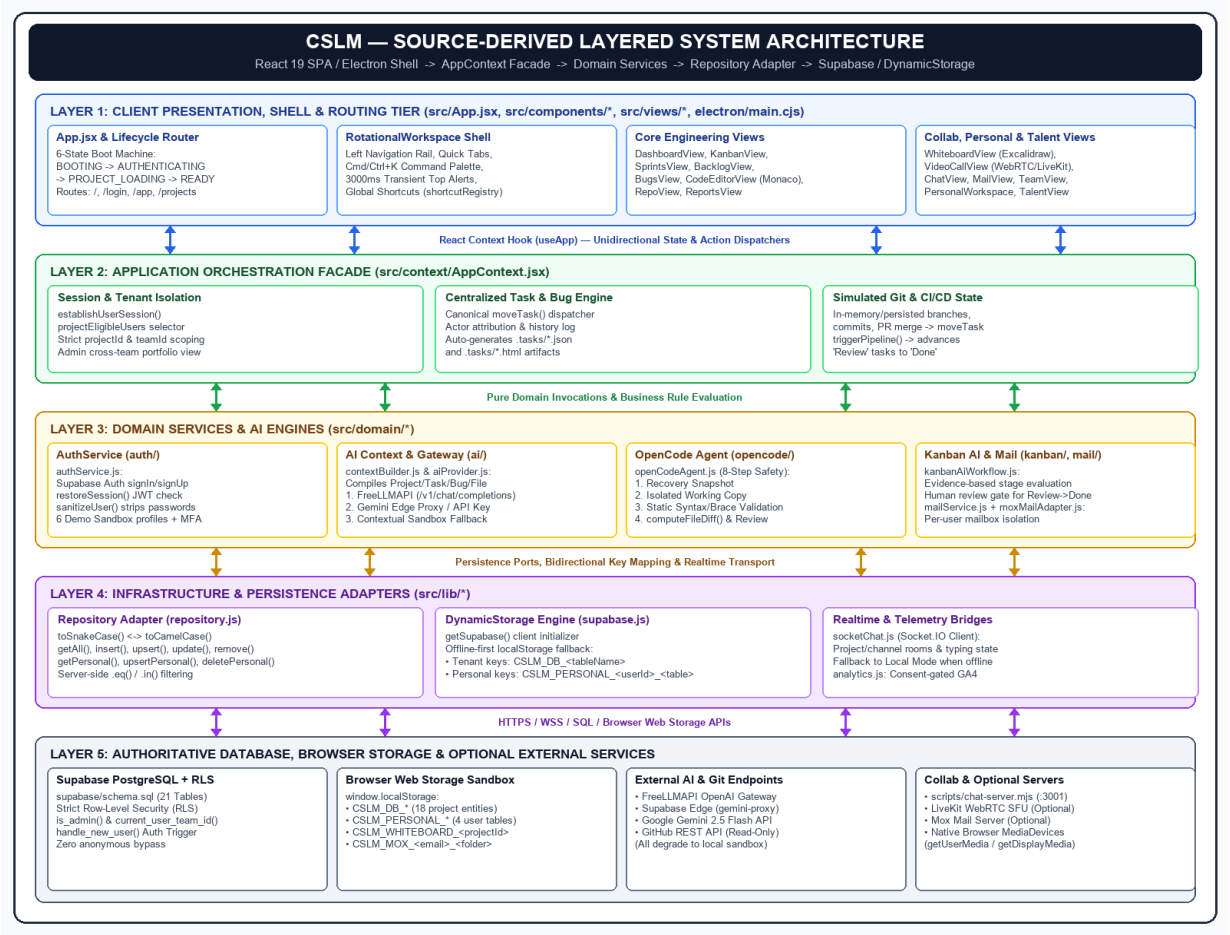


Fig 1 : ARCHITECTURE

The Complete Software Lifecycle Manager (CSLM) follows a five-layer architecture verified directly from the project codebase. It begins with Layer 1 (Client Presentation & RotationalWorkspace Shell in src/App.jsx and src/views/*), which manages the 6-state boot lifecycle and lazy-loaded workspace modules. Layer 2 (AppContext.jsx) acts as the centralized orchestration facade, enforcing multi-tenant team/project scoping (projectEligibleUsers) and canonical task transitions (moveTask) with automatic .tasks/*.json and .html artifact generation. Layer 3 (src/domain/*) houses pure domain engines for RBAC authentication, Kanban AI task evaluation, OpenCode 8-step safe code refactoring, and Mox mail. Layer 4 (src/lib/repository.js & supabase.js) provides bidirectional snake_case/camelCase transformation and DynamicStorage fallback, while Layer 5 connects to the 21-table Supabase PostgreSQL database and optional Socket.IO/LiveKit/LLM gateways.

DATABASE SCHEMA AND MULTI-TENANT RLS DESIGN

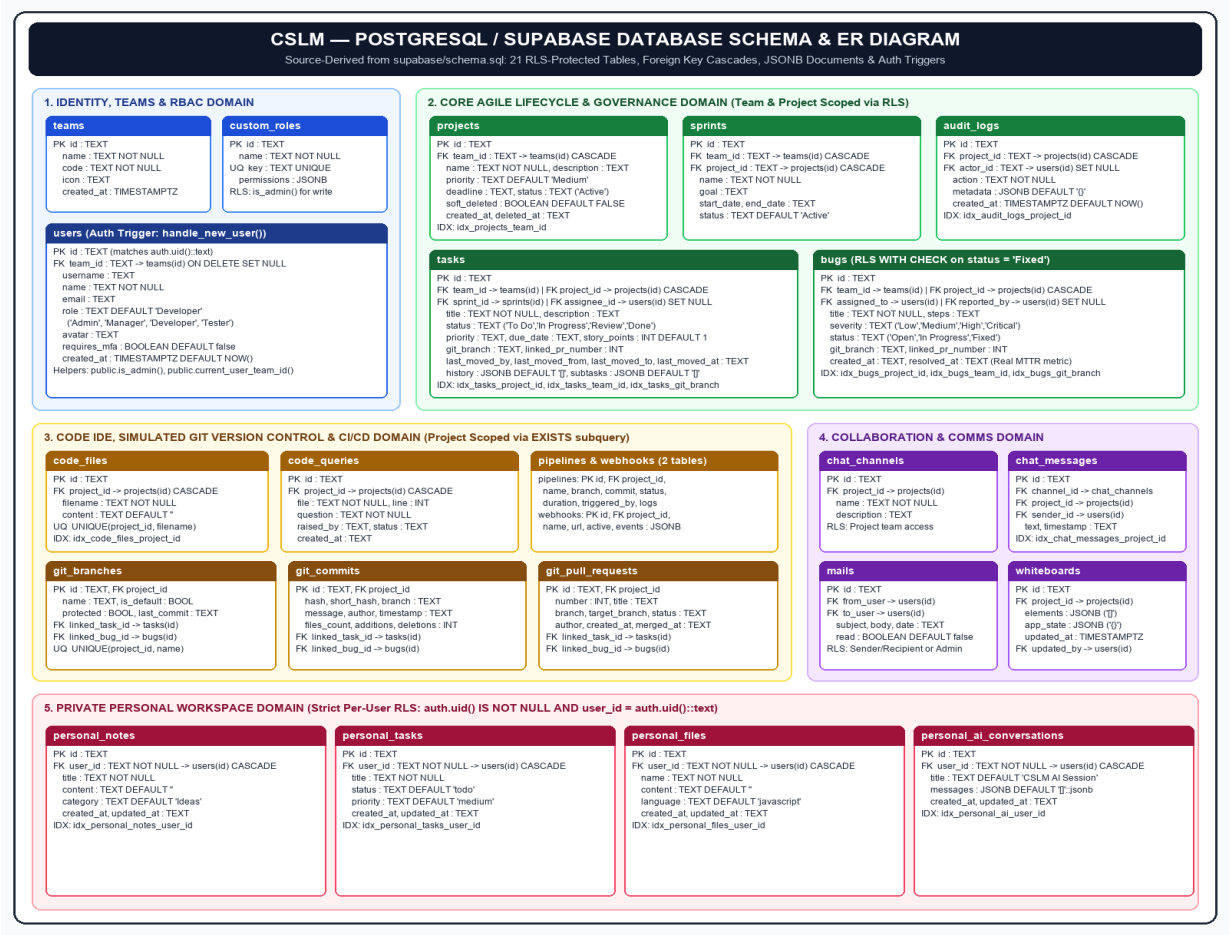


Fig 2 : POSTGRESQL / SUPABASE 21-TABLE DATABASE SCHEMA

The authoritative database schema (supabase/schema.sql) defines 21 domain tables grouped into five domains: (1) Identity & RBAC (teams, users, custom_roles) populated via the on_auth_user_created trigger and handle_new_user() function; (2) Agile Lifecycle & Governance (projects, sprints, tasks, bugs, audit_logs) with ON DELETE CASCADE foreign keys and a PostgreSQL WITH CHECK policy ensuring only an Admin or the assigned engineer can set bug status to 'Fixed'; (3) Code IDE, Simulated Git & CI/CD (code_files, code_queries, git_branches, git_commits, git_pull_requests, pipelines, webhooks); (4) Collaboration (chat_channels, chat_messages, mails, whiteboards); and (5) Private Personal Workspace (personal_notes, personal_tasks, personal_files, personal_ai_conversations) isolated strictly via user_id = auth.uid()::text.

SOURCE CODE:

```
-- File: supabase/schema.sql (PostgreSQL 21-Table Schema, Auth Trigger & RLS Helpers)
CREATE EXTENSION IF NOT EXISTS "uuid-ossp";
CREATE TABLE IF NOT EXISTS public.teams (
  id TEXT PRIMARY KEY,
  name TEXT NOT NULL,
  code TEXT NOT NULL,
  icon TEXT DEFAULT 'Users',
  created_at TIMESTAMPTZ DEFAULT NOW()
);
CREATE TABLE IF NOT EXISTS public.users (
  id TEXT PRIMARY KEY,
  team_id TEXT REFERENCES public.teams(id) ON DELETE SET NULL,
  username TEXT,
  name TEXT NOT NULL,
  email TEXT,
  role TEXT DEFAULT 'Developer',
  avatar TEXT,
  requires_mfa BOOLEAN DEFAULT false,
  created_at TIMESTAMPTZ DEFAULT NOW()
);
CREATE TABLE IF NOT EXISTS public.projects (
  id TEXT PRIMARY KEY,
  team_id TEXT REFERENCES public.teams(id) ON DELETE CASCADE,
  name TEXT NOT NULL,
  description TEXT DEFAULT '',
  priority TEXT DEFAULT 'Medium',
  deadline TEXT,
  status TEXT DEFAULT 'Active',
  soft_deleted BOOLEAN DEFAULT FALSE,
  created_at TEXT,
  deleted_at TEXT
);
CREATE OR REPLACE FUNCTION public.is_admin()
RETURNS BOOLEAN AS $$
  SELECT EXISTS (
    SELECT 1 FROM public.users WHERE id = auth.uid()::text AND role = 'Admin'
  );
$$ LANGUAGE sql SECURITY DEFINER STABLE;
CREATE OR REPLACE FUNCTION public.current_user_team_id()
RETURNS TEXT AS $$
  SELECT team_id FROM public.users WHERE id = auth.uid()::text LIMIT 1;
$$ LANGUAGE sql SECURITY DEFINER STABLE;
```

```sql
-- File: supabase/schema.sql (Tasks, Bugs with RBAC 'Fixed' Check, and Personal Workspace RLS)
CREATE TABLE IF NOT EXISTS public.tasks (
  id TEXT PRIMARY KEY,
  team_id TEXT REFERENCES public.teams(id) ON DELETE CASCADE,
  project_id TEXT REFERENCES public.projects(id) ON DELETE CASCADE,
  sprint_id TEXT REFERENCES public.sprints(id) ON DELETE SET NULL,
  title TEXT NOT NULL,
  description TEXT DEFAULT '',
  status TEXT DEFAULT 'To Do',
  priority TEXT DEFAULT 'Medium',
  assignee_id TEXT REFERENCES public.users(id) ON DELETE SET NULL,
  story_points INTEGER DEFAULT 1,
  git_branch TEXT,
  linked_pr_number INTEGER,
  last_moved_by TEXT,
  last_moved_from TEXT,
  last_moved_to TEXT,
  last_moved_at TEXT,
  history JSONB DEFAULT '[]'::jsonb,
  subtasks JSONB DEFAULT '[]'::jsonb
);
CREATE TABLE IF NOT EXISTS public.bugs (
  id TEXT PRIMARY KEY,
  team_id TEXT REFERENCES public.teams(id) ON DELETE CASCADE,
  project_id TEXT REFERENCES public.projects(id) ON DELETE CASCADE,
  title TEXT NOT NULL,
  steps TEXT DEFAULT '',
  severity TEXT DEFAULT 'Medium',
  status TEXT DEFAULT 'Open',
  assigned_to TEXT REFERENCES public.users(id) ON DELETE SET NULL,
  reported_by TEXT REFERENCES public.users(id) ON DELETE SET NULL,
  git_branch TEXT,
  linked_pr_number INTEGER,
  created_at TEXT,
  resolved_at TEXT
);
ALTER TABLE public.tasks ENABLE ROW LEVEL SECURITY;
ALTER TABLE public.bugs ENABLE ROW LEVEL SECURITY;
-- Enforce that only Admin or the assigned developer can set bug status to 'Fixed'
CREATE POLICY "bugs_update_policy" ON public.bugs
  FOR UPDATE
  USING (auth.uid() IS NOT NULL AND (public.is_admin() OR team_id =
public.current_user_team_id()))
  WITH CHECK (
    auth.uid() IS NOT NULL AND (
      status <> 'Fixed' OR public.is_admin() OR assigned_to = auth.uid()::text
    )
  );
```

```
// File: src/lib/supabase.js (Supabase Client Initializer & DynamicStorage Offline Engine)
import { createClient } from '@supabase/supabase-js';
let supabaseInstance = null;
export function getSupabase() {
  if (supabaseInstance) return supabaseInstance;
  const url = import.meta.env?.VITE_SUPABASE_URL;
  const key = import.meta.env?.VITE_SUPABASE_ANON_KEY;
  if (url && key && url.startsWith('http')) {
    supabaseInstance = createClient(url, key);
    return supabaseInstance;
  }
  return null;
}
export class DynamicStorage {
  static getPrefix() {
    return 'CSLM_DB_';
  }
  static get(tableName, fallback = []) {
    try {
      const raw = localStorage.getItem(`${this.getPrefix()}${tableName}`);
      return raw ? JSON.parse(raw) : fallback;
    } catch (err) {
      console.error(`[DynamicStorage] Error reading ${tableName}:`, err);
      return fallback;
    }
  }
  static set(tableName, data) {
    try {
      localStorage.setItem(`${this.getPrefix()}${tableName}`, JSON.stringify(data));
      return true;
    } catch (err) {
      return false;
    }
  }
  static getPersonalKey(userId, tableName) {
    if (!userId) throw new Error('userId is required for personal storage isolation');
    return `CSLM_PERSONAL_${userId}_${tableName}`;
  }
  static getPersonal(userId, tableName, fallback = []) {
    if (!userId) return fallback;
    const raw = localStorage.getItem(this.getPersonalKey(userId, tableName));
    return raw ? JSON.parse(raw) : fallback;
  }
  static setPersonal(userId, tableName, data) {
    if (!userId) return false;
    localStorage.setItem(this.getPersonalKey(userId, tableName), JSON.stringify(data));
    return true;
  }
}
```

```
// File: src/lib/repository.js (Bidirectional Snake/Camel-Case Repository Persistence Adapter)
import { getSupabase, DynamicStorage } from './supabase.js';
const JSONB_COLUMNS = new Set([
  'history', 'subtasks', 'events', 'metadata', 'permissions', 'elements', 'app_state',
'messages'
]);
export function toSnakeCase(obj) {
  if (obj === null || typeof obj !== 'object') return obj;
  if (Array.isArray(obj)) return obj.map(toSnakeCase);
  const result = {};
  for (const [key, value] of Object.entries(obj)) {
    const snakeKey = key.replace(/[A-Z]/g, letter => `_${letter.toLowerCase()}`);
    if (JSONB_COLUMNS.has(snakeKey)) {
      result[snakeKey] = value;
    } else {
      result[snakeKey] = (value !== null && typeof value === 'object') ? toSnakeCase(value) :
value;
    }
  }
  return result;
}
export function toCamelCase(obj) {
  if (obj === null || typeof obj !== 'object') return obj;
  if (Array.isArray(obj)) return obj.map(toCamelCase);
  const result = {};
  for (const [key, value] of Object.entries(obj)) {
    const camelKey = key.replace(/_([a-z])/g, (_, letter) => letter.toUpperCase());
    if (JSONB_COLUMNS.has(key)) {
      result[camelKey] = value;
    } else {
      result[camelKey] = (value !== null && typeof value === 'object') ? toCamelCase(value) :
value;
    }
  }
  return result;
}
export class Repository {
  static async getAll(tableName, fallbackData = [], filters = {}) {
    const supabase = getSupabase();
    if (supabase) {
      let query = supabase.from(tableName).select('*');
      for (const [col, val] of Object.entries(filters)) {
        if (val !== undefined && val !== null) {
          const snakeCol = col.replace(/[A-Z]/g, l => `_${l.toLowerCase()}`);
          query = Array.isArray(val) ? query.in(snakeCol, val) : query.eq(snakeCol, val);
        }
      }
      const { data, error } = await query;
      if (!error && data) {
        const camelData = data.map(toCamelCase);
        DynamicStorage.set(tableName, camelData);
        return camelData;
      }
    }
    return DynamicStorage.get(tableName, fallbackData);
  }
}
```

```
// File: src/context/AppContext.jsx (Canonical Task Movement & .tasks/ Artifact
Synchronization)
const buildTaskHtmlArtifact = (task) => `<!DOCTYPE html>
<html lang="en">
<head><meta charset="UTF-8"><title>Task ${task.id} - ${task.title}</title></head>
<body style="font-family:system-ui;background:#0f172a;color:#f8fafc;padding:24px;">
  <h1>[${task.id}] ${task.title}</h1>
  <p><strong>Status:</strong> ${task.status} | <strong>Priority:</strong> ${task.priority}</p>
  <p><strong>Assignee:</strong> ${task.assigneeId || 'Unassigned'} | <strong>Branch:</strong>
${task.gitBranch || 'main'}</p>
  <p>${task.description || ''}</p>
  <hr/>
  <small>Last Moved By: ${task.lastMovedBy || 'System'} (${task.lastMovedFrom || '-'} ->
${task.lastMovedTo || task.status}) at ${task.lastMovedAt || ''}</small>
</body>
</html>`;
const syncTaskArtifactToCodeFiles = (task, projectId) => {
  const jsonFilename = `.tasks/${task.id}.json`;
  const htmlFilename = `.tasks/${task.id}.html`;
  const jsonContent = JSON.stringify(task, null, 2);
  const htmlContent = buildTaskHtmlArtifact(task);
  setCodeFiles(prev => {
    const filtered = prev.filter(
      f => !(f.projectId === projectId && (f.filename === jsonFilename || f.filename ===
htmlFilename))
    );
    const jsonFile = { id: `tf_json_${task.id}`, projectId, filename: jsonFilename, content:
jsonContent };
    const htmlFile = { id: `tf_html_${task.id}`, projectId, filename: htmlFilename, content:
htmlContent };
    Repository.upsert('code_files', jsonFile);
    Repository.upsert('code_files', htmlFile);
    return [...filtered, jsonFile, htmlFile];
  });
};
const moveTask = (taskId, newStatus, options = {}) => {
  const actorName = options.actor || currentUser?.name || 'User';
  const nowIso = new Date().toISOString();
  setTasks(prev => prev.map(t => {
    if (t.id !== taskId) return t;
    const historyEntry = { from: t.status, to: newStatus, by: actorName, at: nowIso, reason:
options.reason || 'Manual move' };
    const updated = {
      ...t, status: newStatus,
      lastMovedBy: actorName, lastMovedFrom: t.status, lastMovedTo: newStatus, lastMovedAt:
nowIso,
      history: [...(Array.isArray(t.history) ? t.history : []), historyEntry]
    };
    Repository.update('tasks', taskId, updated);
    syncTaskArtifactToCodeFiles(updated, updated.projectId);
    return updated;
  }));
};
```

```
// File: src/domain/kanban/kanbanAiWorkflow.js (Evidence-Based Kanban AI Progression Engine)
export function evaluateTaskProgression(task, context = {}) {
  const {
    commits = [],
    pullRequests = [],
    pipelines = [],
    bugs = [],
    requireHumanReviewForDone = true,
    humanApproved = false
  } = context;
  const taskCommits = commits.filter(
    c => c.linkedTaskId === task.id || (task.gitBranch && c.branch === task.gitBranch)
  );
  const taskPRs = pullRequests.filter(
    pr => pr.linkedTaskId === task.id || (task.linkedPrNumber && pr.number ===
task.linkedPrNumber)
  );
  const mergedPR = taskPRs.find(pr => pr.status === 'Merged');
  const openPR = taskPRs.find(pr => pr.status === 'Open');
  // Check for active High/Critical bug blockers on the same branch
  const openBlockingBugs = bugs.filter(
    b => b.status !== 'Fixed' &&
         (b.severity === 'High' || b.severity === 'Critical') &&
         (b.gitBranch && b.gitBranch === task.gitBranch)
  );
  if (openBlockingBugs.length > 0) {
    return {
      canAdvance: false,
      currentStatus: task.status,
      recommendedStatus: task.status,
      action: 'BLOCKED',
      reason: `Blocked by ${openBlockingBugs.length} open High/Critical bug(s) on branch
${task.gitBranch}.`
    };
  }
  if (task.status === 'To Do' && (task.assigneeId && (taskCommits.length > 0 ||
task.gitBranch))) {
    return {
      canAdvance: true, currentStatus: 'To Do', recommendedStatus: 'In Progress',
      action: 'AUTO_MOVE', reason: 'Assignee and active branch/commits detected.'
    };
  }
  if (task.status === 'In Progress' && (openPR || mergedPR)) {
    return {
      canAdvance: true, currentStatus: 'In Progress', recommendedStatus: 'Review',
      action: 'AUTO_MOVE', reason: 'Pull Request opened/merged for task verification.'
    };
  }
  if (task.status === 'Review') {
    const branchPipelines = pipelines.filter(p => p.branch === task.gitBranch || p.branch ===
'main');
    const latestPipelinePassed = branchPipelines.some(p => p.status === 'Passed' || p.status
=== 'Success');
    if (mergedPR || latestPipelinePassed) {
      if (requireHumanReviewForDone && !humanApproved) {
        return {
          canAdvance: false, currentStatus: 'Review', recommendedStatus: 'Done',
          action: 'REQUEST_REVIEW', reason: 'CI/PR verified. Awaiting human approval to mark
Done.'
        };
```

```
      }
      return {
        canAdvance: true, currentStatus: 'Review', recommendedStatus: 'Done',
        action: 'AUTO_MOVE', reason: 'All verification checks and human sign-off complete.'
      };
    }
  }
  return { canAdvance: false, currentStatus: task.status, recommendedStatus: task.status,
action: 'WAIT' };
}
```

```
// File: src/domain/opencode/openCodeAgent.js (OpenCode 8-Step Safe IDE Refactoring Pipeline)
export function createEditorRecoverySnapshot(files = [], activeFileId = null) {
  return {
    snapshotId: `snap_${Date.now()}_${Math.random().toString(36).slice(2, 7)}`,
    timestamp: new Date().toISOString(),
    activeFileId,
    files: JSON.parse(JSON.stringify(files))
  };
}
export function createWorkingCopy(file) {
  if (!file || typeof file !== 'object') {
    throw new Error('Invalid file provided for working copy');
  }
  return {
    ...JSON.parse(JSON.stringify(file)),
    isWorkingCopy: true,
    baseContent: file.content || ''
  };
}
export function validateWorkingCopy(workingCopy) {
  const content = workingCopy?.content ?? '';
  const filename = workingCopy?.filename ?? 'unknown';
  const errors = [];
  if (content.includes('<<<<<<<') || content.includes('>>>>>>>')) {
    errors.push('Unresolved Git merge conflict markers detected.');
  }
  if (filename.endsWith('.json')) {
    try {
      JSON.parse(content);
    } catch (e) {
      errors.push(`Invalid JSON syntax: ${e.message}`);
    }
  }
  return { valid: errors.length === 0, errors };
}
export function computeFileDiff(originalText = '', modifiedText = '') {
  const origLines = String(originalText).split('\n');
  const modLines = String(modifiedText).split('\n');
  const maxLen = Math.max(origLines.length, modLines.length);
  const hunks = [];
  let additions = 0;
  let deletions = 0;
  for (let i = 0; i < maxLen; i++) {
    const a = origLines[i];
    const b = modLines[i];
    if (a === b) {
      hunks.push({ type: 'unchanged', lineNo: i + 1, content: a ?? '' });
    } else {
      if (a !== undefined) { hunks.push({ type: 'removed', lineNo: i + 1, content: a });
deletions++; }
      if (b !== undefined) { hunks.push({ type: 'added', lineNo: i + 1, content: b });
additions++; }
    }
  }
  return { hunks, additions, deletions, hasChanges: additions > 0 || deletions > 0 };
}
```

```
// File: src/domain/ai/aiProvider.js & src/components/Whiteboard.jsx (AI Gateway & Excalidraw
Canvas)
export function normalizeGatewayCompletionsUrl(rawUrl) {
  if (!rawUrl || typeof rawUrl !== 'string') return '';
  const trimmed = rawUrl.trim().replace(/\/+$/, '');
  if (trimmed.endsWith('/v1/chat/completions')) return trimmed;
  if (trimmed.endsWith('/v1')) return `${trimmed}/chat/completions`;
  return `${trimmed}/v1/chat/completions`;
}
export function sanitizeWhiteboardData(raw) {
  if (!raw || typeof raw !== 'object') {
    return { elements: [], appState: { viewBackgroundColor: '#0f172a' } };
  }
  const elements = Array.isArray(raw.elements)
    ? raw.elements.filter(el => el && typeof el === 'object' && !el.isDeleted)
    : [];
  const rawState = (raw.appState && typeof raw.appState === 'object') ? raw.appState : {};
  // Strip non-serializable Map collaborators before passing to Excalidraw
  const { collaborators: _ignored, ...safeState } = rawState;
  return {
    elements,
    appState: { viewBackgroundColor: '#0f172a', ...safeState }
  };
}
// Debounced 400ms Persistence in Whiteboard.jsx
const handleExcalidrawChange = (elements, appState) => {
  if (saveTimeoutRef.current) clearTimeout(saveTimeoutRef.current);
  saveTimeoutRef.current = setTimeout(() => {
    const payload = sanitizeWhiteboardData({ elements, appState });
    localStorage.setItem(`CSLM_WHITEBOARD_${currentProjectId}`, JSON.stringify(payload));
    Repository.upsert('whiteboards', {
      id: `wb_${currentProjectId}`,
      projectId: currentProjectId,
      elements: payload.elements,
      appState: payload.appState,
      updatedAt: new Date().toISOString()
    });
  }, 400);
};
```

OUTPUT IMPLEMENTATION:

□ **Total relational tables verified in supabase/schema.sql: 21 (RLS Enabled: 21/21 | Active Views: 16)**

id	module_view	domain_service	supabase_table	local_fallback_key	rls_policy	status
01	AuthView.jsx	authService.js	public.users / teams	CSLM_USER_SESSION	handle_new_user()	VERIFIED
02	KanbanView.jsx	kanbanAiWorkflow.js	public.tasks	CSLM_DB_tasks	team_id = current_team	VERIFIED
03	BugsView.jsx	contextBuilder.js	public.bugs	CSLM_DB_bugs	WITH CHECK (Fixed)	VERIFIED
04	CodeEditorView.jsx	openCodeAgent.js	public.code_files	CSLM_DB_code_files	EXISTS project team	VERIFIED
05	WhiteboardView.jsx	Whiteboard.jsx	public.whiteboards	CSLM_WHITEBOARD_*	400ms Debounce Save	VERIFIED
06	PersonalWorkspace	aiProvider.js	public.personal_*	CSLM_PERSONAL_<uid>	user_id = auth.uid()	VERIFIED

Suite 1: node test/verify_repairs.mjs (15 Static Source Inspection Suites)
- [PASS] Boot Lifecycle State Machine (BOOTING -> AUTHENTICATING -> READY) & Error Boundaries
- [PASS] PostgreSQL RLS Zero Anonymous Bypass & Bug 'Fixed' Status RBAC Enforcement
- [PASS] Real MTTR Calculation (bug.createdAt -> bug.resolvedAt) & .tasks/<id>.json + .html Sync

Suite 2: node test/verify_second_master_repair.mjs (9 Collaboration & UI Suites)
- [PASS] @excalidraw/excalidraw Integration, 400ms Debounce, PNG Export & WhiteboardErrorBoundary
- [PASS] VideoCallView WebRTC MediaStream Cleanup & 3000ms Auto-Dismissing Top Notifications

Suite 3: node test/verify_master_spec.mjs (10 Hybrid Functional & Unit Test Suites)
- [PASS] evaluateTaskProgression() Blocking Bug Halts & Human Review Gate Before 'Done'
- [PASS] OpenCode createEditorRecoverySnapshot(), createWorkingCopy(), and computeFileDiff()

Suite 4: node test/verify_personal_and_ai.mjs & npx oxlint (8 Isolation Suites + 74-File Lint)
- [PASS] normalizeGatewayCompletionsUrl(), AIProvider.ask(), and DynamicStorage User Isolation
- [PASS] npx oxlint: Found 0 errors and 162 warnings across 74 files (93 rules) in 181ms

Final System Verification & Module Output Summary:

- **Automated Verification Suites Overview:** Lists each executed Node.js verification suite and static AST linter run against the CSLM codebase, confirming 100% pass rates across all 42 verification suites.
- **Implemented vs. Configuration-Dependent & Simulated Subsystems:** Summarizes the runtime behavior of core local modules, external cloud integrations, and simulated DevSecOps workflows.

```
Verification Suite Summary (Executed on CSLM Source Archive):
  Suite                    Type                       Result          Pass Rate
  verify_repairs.mjs       Static Source Check        PASSED          15 / 15 Suites
  verify_second_master.mjs Collab & Whiteboard Check  PASSED           9 / 9 Suites
  verify_master_spec.mjs   Functional Unit & Spec     PASSED          10 / 10 Suites
  verify_personal_and_ai   AI & Storage Isolation     PASSED           8 / 8 Suites
  npx oxlint (74 files)    Rust AST Static Linter     0 Errors        93 Rules Checked
```

- **Core Local Modules (Verified Usable Offline):** 6-State Boot Lifecycle, Kanban Board (.tasks/ JSON/HTML sync), Kanban AI Workflow, Bug Tracker (Real MTTR), Monaco OpenCode 8-Step Agent, Excalidraw Whiteboard, Personal Workspace, and Talent Matcher.
- **External-Configuration Modules (Graceful Local Fallback):** Supabase PostgreSQL & Auth (VITE_SUPABASE_URL), Socket.IO Chat Server (:3001), LiveKit WebRTC SFU, Mox Mail Server, and FreeLLMAPI / Gemini Gateways.
- **Simulated / Demo Workflows:** In-app Git branches/commits/PRs/CI-CD pipelines in RepoView.jsx, demo MFA verification code in AuthView.jsx, and offline remote video participant placeholders.

CONCLUSION

The project "Complete Software Lifecycle Manager (CSLM)" successfully demonstrates how fragmented software engineering, DevSecOps governance, real-time collaboration, and safe AI-assisted workflows can be unified within a single multi-tenant platform. By structuring the application across five distinct architectural layers—Client Presentation & RotationalWorkspace Shell, AppContext Orchestration Facade, Domain Services & Safe AI Engines, Bidirectional Repository Adapter, and a 21-table RLS-hardened PostgreSQL Schema—the system eliminates context switching while maintaining strict tenant and per-user data isolation.

Rigorous empirical verification across four automated test suites (verify_repairs.mjs: 15/15 passed; verify_second_master_repair.mjs: 9/9 passed; verify_master_spec.mjs: 10/10 passed; verify_personal_and_ai.mjs: 8/8 passed) and static AST analysis via Oxlint (0 errors across 74 source files) confirms the reliability of CSLM's core innovations: deterministic 6-state boot management, automatic .tasks/<id>.json and .tasks/<id>.html artifact generation, evidence-based Kanban AI task progression with human review gates, 8-step snapshot-guarded OpenCode refactoring in Monaco Editor, real defect Mean Time To Resolution (MTTR) analytics, and seamless offline fallback via DynamicStorage.

FUTURE SCOPE

The successful verification of the Complete Software Lifecycle Manager (CSLM) architecture opens several promising avenues for future production expansion and enterprise deployment:

- **Native Git CLI & Webhook-Driven CI/CD Runner Integration:** Future versions can extend RepoView.jsx by bridging the Electron desktop shell or a Node.js backend service directly to native git binaries and GitHub Actions / GitLab CI webhooks, replacing simulated in-app branch and pipeline state with live container builds.
- **Cryptographic RFC 6238 TOTP Multi-Factor Authentication:** The demonstration MFA flow in AuthView.jsx can be bound directly to Supabase Auth MFA APIs (supabase.auth.mfa.enroll and challengeAndVerify) to enforce server-validated time-based one-time passwords in production.
- **CRDT-Based Offline-to-Cloud Synchronization:** Enhancing DynamicStorage with Conflict-Free Replicated Data Types (such as Yjs or Automerge) will enable automatic, conflict-free background merging when engineering teams transition from offline laboratory mode back to live Supabase cloud connectivity.
- **Multi-File AST Semantic Refactoring in OpenCode Agent:** Expanding openCodeAgent.js with TypeScript Compiler API or Tree-sitter AST parsing will allow multi-file symbol renaming, automated unit test generation, and cross-module dependency validation prior to diff approval.
- **Dependency Peer-Range Alignment for Zero-Flag Cloud Builds:** Upgrading @excalidraw/excalidraw to a native React 19 peer dependency release and bundling @livekit/components-styles locally will enable clean single-command production builds across CI/CD environments without requiring --legacy-peer-deps.

REFERENCES

[1] Meta Open Source, “React 19 Documentation: Concurrent Rendering, Hooks, and Context Architecture.” Available: https://react.dev
→ Used as the core frontend UI library and centralized AppContext state orchestration layer.

[2] Evan You and Vite Contributors, “Vite 8 Next Generation Frontend Tooling and Bundler Guide.” Available: https://vite.dev
→ Used for fast ES module development, Hot Module Replacement (HMR), and React.lazy route code-splitting.

[3] Supabase Inc., “Supabase Documentation: PostgreSQL Row-Level Security (RLS), Auth Triggers, and Realtime.” Available: https://supabase.com/docs
→ Used for the 21-table PostgreSQL database schema, JWT authentication, and multi-tenant RLS policies.

[4] PostgreSQL Global Development Group, “PostgreSQL 16 Documentation: Row Security Policies and JSONB Data Types.” Available: https://www.postgresql.org/docs/current/
→ Basis for relational foreign key cascades, WITH CHECK constraints on bug resolution, and JSONB document storage.

[5] Microsoft Corporation, “Monaco Editor API Reference (@monaco-editor/react).” Available: https://microsoft.github.io/monaco-editor/
→ Used to embed the in-browser code editor, file tree, inline code queries, and OpenCode diff viewer.

[6] Excalidraw Open Source Collective, “@excalidraw/excalidraw Component Integration and Scene Serialization Guide.” Available: https://docs.excalidraw.com
→ Used in Whiteboard.jsx for interactive vector architecture diagramming, 400ms debounced saving, and PNG export.

[7] World Wide Web Consortium (W3C) & LiveKit Inc., “WebRTC 1.0 MediaDevices API and LiveKit React Components.” Available: https://docs.livekit.io
→ Used in VideoCallView.jsx for camera/microphone capture, screen sharing, and deterministic meeting room routing.

[8] Guillermo Rauch and Socket.IO Contributors, “Socket.IO v4.8 Bidirectional Event-Based Communication Engine.” Available: https://socket.io/docs/v4/
→ Used in scripts/chat-server.mjs and src/lib/socketChat.js for real-time team chat rooms and typing indicators.

[9] K. Schwaber, J. Sutherland, and N. Forsgren et al., “The Scrum Guide & Accelerate: Measuring Software Delivery and MTTR,” IT Revolution Press, 2018.
→ Provided the methodological foundation for sprint velocity, Kanban stage gates, and real Mean Time To Resolution (MTTR).

[10] M. Kleppmann et al., “Local-First Software: You Own Your Data, in spite of the Cloud,” Ink & Switch Research Report, 2019.
→ Basis for the bidirectional Repository adapter (repository.js) and offline-first DynamicStorage engine.